

DUCHO: A Unified Framework for the Extraction of Multimodal Features in Recommendation

Daniele Malitesta*
Politecnico di Bari, Italy
daniele.malitesta@poliba.it

Giuseppe Gassi*
Politecnico di Bari, Italy
g.gassi@studenti.poliba.it

Claudio Pomo
Politecnico di Bari, Italy
claudio.pomo@poliba.it

Tommaso Di Noia
Politecnico di Bari, Italy
tommaso.dinoia@poliba.it

ABSTRACT

In multimodal-aware recommendation, the extraction of meaningful multimodal features is at the basis of high-quality recommendations. Generally, each recommendation framework implements its multimodal extraction procedures with specific strategies and tools. This is limiting for two reasons: (i) different extraction strategies do not ease the interdependence among multimodal recommendation frameworks; thus, they cannot be efficiently and fairly compared; (ii) given the large plethora of pre-trained deep learning models made available by different open source tools, model designers do not have access to shared interfaces to extract features. Motivated by the outlined aspects, we propose **DUCHO**, a unified framework for the extraction of multimodal features in recommendation. By integrating three widely-adopted deep learning libraries as backends, namely, TensorFlow, PyTorch, and Transformers, we provide a shared interface to extract and process features where each backend's specific methods are abstracted to the end user. Noteworthy, the extraction pipeline is easily configurable with a YAML-based file where the user can specify, for each modality, the list of models (and their specific backends/parameters) to perform the extraction. Finally, to make **DUCHO** accessible to the community, we build a public Docker image equipped with a ready-to-use CUDA environment and propose three demos to test its functionalities for different scenarios and tasks. The GitHub repository and the documentation are accessible at this link: <https://github.com/sisinflab/Ducho>.

CCS CONCEPTS

• **Information systems** → **Multimedia and multimodal retrieval**; **Personalization**.

KEYWORDS

Multimodal Recommendation, Deep Neural Networks

ACM Reference Format:

Daniele Malitesta, Giuseppe Gassi, Claudio Pomo, and Tommaso Di Noia. 2023. **DUCHO: A Unified Framework for the Extraction of Multimodal Features in Recommendation**. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23)*, October 29–November 3, 2023, Ottawa, ON, Canada. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3581783.3613458>

*Corresponding authors: Daniele Malitesta (daniele.malitesta@poliba.it) and Giuseppe Gassi (g.gassi@studenti.poliba.it).



This work is licensed under a Creative Commons Attribution International 4.0 License.

MM '23, October 29–November 3, 2023, Ottawa, ON, Canada
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0108-5/23/10.
<https://doi.org/10.1145/3581783.3613458>

1 INTRODUCTION AND MOTIVATION

With the advent of the digital era and the Internet, numerous online services have emerged, including platforms for e-commerce, media streaming, and social networks. The vast majority of such websites rely on recommendation algorithms to provide users with a personalized surfing experience. In specific domains such as fashion [3], music [6], food [5], and micro-video [8] recommendation, recommender systems have demonstrated to be effectively supported in their decision-making process by all types of multimodal data sources the users usually interact with (e.g., product images and descriptions, users' reviews, audio tracks).

The literature refers to multimodal-aware recommender systems (MRSs) as the family of recommendation algorithms leveraging multimodal (i.e., audio, visual, textual) content data to augment the representation of items, thus tackling issues in the field such as the sparsity of the user-item matrix and the inexplicable nature of users' actions (e.g., clicks, views) on online platforms which may not always be easy to profile for the recommendation algorithms.

Despite being the initial stage of any multimodal recommendation pipeline, the extraction of meaningful multimodal features is paramount in delivering high-quality recommendations [2]. However, the current practice of employing diverse multimodal extraction procedures in each recommendation framework poses limitations. Firstly, these diverse implementations hinder the interdependence across various multimodal recommendation frameworks, making their fair comparison difficult [4]. Secondly, despite the availability of numerous pre-trained deep learning models in popular open source libraries, the lack of shared interfaces for feature extraction across them represents a challenge for model designers.

To address these shortcomings, we propose **DUCHO**, a unified framework designed to streamline the extraction of multimodal features for recommendation systems. By integrating widely-adopted deep learning libraries as backends such as TensorFlow, PyTorch, and Transformers, we establish a shared interface that empowers users to extract and process audio, visual, and textual features from both items and user-item interactions (see Table 1). This abstraction allows to leverage methods from each backend without being encumbered by the specific implementation that backend poses. A notable feature of our framework lays in its easily configurable extraction pipeline, which can be personalized using a YAML-based file. Users can specify the desired models, their respective backends, and models' parameters (such as the extraction layer).

By looking at the related literature, the most similar application to **DUCHO** is Cornac [7], a framework for multimodal-aware recommendation. For the sake of completeness, we report their main differences. Differently from Cornac, **DUCHO**: (i) is specifically aimed to provide customizable multimodal feature extractions, being completely agnostic to the downstream recommender system

Table 1: An overview of all modalities, sources, and backends combinations available in DUCHO.

Modalities	Sources		Backends		
	Items	Interactions	TensorFlow	PyTorch	Transformers
Audio	✓	✓		✓	✓
Visual	✓	✓	✓	✓	
Textual	✓	✓			✓

that might exploit the extracted features, thus being easily applicable to any model; (ii) provides the user with the possibility to select the deep learning extraction model, its backend, and its output layer; (iii) introduces the audio modality to the modalities set.

To foster the adoption of DUCHO, we also develop a public Docker image pre-equipped with a ready-to-use CUDA environment¹, and propose three demos to show DUCHO’s functionalities. The GitHub repository, which comes with all needed resources is available at: <https://github.com/sisinflab/Ducho>.

2 ARCHITECTURE

DUCHO’s architecture is built upon three main modules, namely, **Dataset**, **Extractor**, and **Runner**, where the first two modules provide different implementations depending on the specific modality (i.e., audio, visual, textual) taken into account. We also remind the **Configuration** one among the other auxiliary components. The architecture is designed to be highly modular, possibly integrating new modules or customizing the existing ones. In the following, we dive deep into each outlined module/component.

2.1 Dataset

The **Dataset** module manages the loading and processing of the input data provided by the user. Starting from a general shared schema for all available modalities, this module provides three separate implementations: **Audio**, **Visual**, and **Textual** Datasets. As a common approach in the literature, the Audio and Visual Datasets require the path to the folder from which image/audio files are loaded, while the Textual Dataset works through a tsv file mapping all the textual characteristics to the inputs.

Noteworthy, and differently from other existing solutions, DUCHO may handle each modality in two fashions, depending on whether the specific modality is describing either the **items** (e.g., product descriptions) or the **interactions** among users and items (e.g., reviews [1]). Concretely, while items are mapped to their unique ids (extracted from the filename or the tsv file), interactions are mapped to the user-item pair (extracted from the tsv file) they refer to. Although the pre-processing and extraction phases do not change at items- and interactions-level (see later), we believe this schema may perfectly suit novel multimodal-aware recommender systems with modalities describing every type of input source (even **users**).

Another important task for the Dataset module is to handle the pre-processing stage of data input. Depending on the specific modality involved, DUCHO offers the possibility to:

- **audio**: load the input audio by extracting the waveform and sample rate, and re-sample it according to the sample rate the pre-trained model was trained on;
- **visual**: convert input images into RGB and resize/normalize them to align with the pre-trained extraction model;
- **textual**: (optionally) clean the input texts to remove or modify noisy textual patterns such as punctuation and digits.

After the extraction phase (see later), the Dataset module is finally in charge of saving the generated multimodal features into **numpy** array format following the file naming scheme from the previous mapping.

2.2 Extractor

The **Extractor** module builds an extraction model from a pre-trained network and works on each loaded/pre-processed input sample to extract its multimodal features. In a similar manner to the Dataset module, the Extractor provides three different implementations for each modality, namely, the **Audio**, **Visual**, and **Textual** Extractors. DUCHO exposes a wide range of pre-trained models from three main backends: TensorFlow, PyTorch, and Transformers. The following modality/backend combinations are currently available:

- **audio**: PyTorch (Torchaudio) and Transformers;
- **visual**: Tensorflow and PyTorch (Torchvision);
- **textual**: Transformers (and SentenceTransformers).

To perform the feature extraction, DUCHO takes as input the (list of) extraction layers for any pre-trained model. Since each backend handles the extraction of hidden layers within a network differently, we follow the guidelines provided in the official documentations, assuming that the user will follow the same naming/indexing scheme of the layers and know the structure of the selected pre-trained model in advance. The interested reader may refer to the README² under the `config/` folder on GitHub for an exhaustive explanation on how to set the extraction layer in each modality/backend setting.

Finally, for the textual case, the user can also specify the specific task the pre-trained model should be trained on (e.g., sentiment analysis), as each pre-trained network may come with different versions depending on the training strategy.

2.3 Runner

The **Runner** module is the orchestrator of DUCHO, whose purpose is to instantiate, call, and manage all the described modules. With its API methods, this module can trigger the complete extraction pipeline (see later) of one single modality or all the modalities involved simultaneously.

The Runner module is conveniently customized through an auxiliary **Configuration** component which stores and exposes all parameters to configure the extraction pipeline. Even if a default configuration is already made available for the user’s sake, DUCHO allows to override some (or all) its parameters through an external configuration file (in YAML format) and/or key-value pairs as input arguments if running the scripts from the command line. Once again, we suggest the readers refer to the README under the `config/` folder on GitHub to understand the general schema of the YAML configuration file.

¹<https://hub.docker.com/r/sisinflabpoliba/ducho>.

²<https://github.com/sisinflab/Ducho/blob/main/config/README.md>.

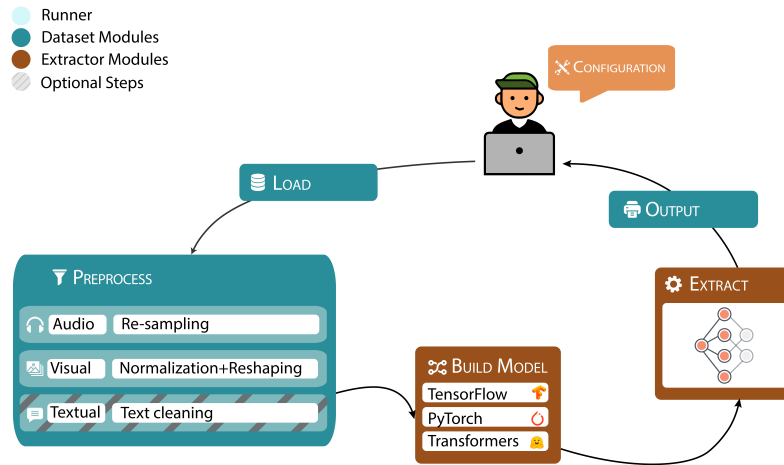


Figure 1: DUCHO's pipeline for multimodal feature extraction, managed by the Dataset, Extractor, and Runner modules.

3 EXTRACTION PIPELINE

The overall multimodal extraction pipeline is represented in Figure 1. Through the Dataset module, the **load** and **preprocess** steps take place, assuming that the user is providing the input data and the YAML configuration file (overridable from command line) to customize the extraction. Then, the Extractor module is in charge of **building** the extraction model(s) by setting the backends and output layer(s). Finally, after the multimodal feature **extraction**, features are saved to the **output** path in numpy format (the Dataset module again controls this latter phase). As previously described, the whole process is orchestrated by the Runner module.

4 DUCHO AS DOCKER APPLICATION

To fully exploit the GPU-speedup implemented in all backends we use for the multimodal feature extraction, one of the basic requirements is to setup a suitable development environment where the backends' versions are compatible with CUDA and, optionally, cuDNN. Generally, setting a workstation where all such libraries/tools are correctly aligned is challenging. To this end, we decide to dockerize DUCHO by making it into a Docker image (available on Docker Hub³) with all packages already installed in a tested and safe virtualization environment on your physical machine.

Our Docker image is built from an NVIDIA-based image which comes with CUDA 11.8 and cuDNN 8 on Ubuntu 22.04, Python 3.8 and Pip, and our cloned repository having all Python packages already installed and ready to be used. A possible container instantiated from the image should specify the gpus to use from the host machine (this feature is currently available on Docker but it depends on the version of CUDA to be installed), and the volume you may want to use to save the framework's outputs permanently.

Note that a generic container instantiated from our image would prompt the user to a shell environment where one could run custom multimodal feature extractions via the command line, and also create custom configuration files for the same purpose.

5 DEMONSTRATIONS

This section proposes three use cases (i.e., demos) which show some of the main functionalities in DUCHO and how to exploit them within a complete multimodal extraction pipeline. The guidelines and codes are accessible at this link⁴ to run the demos (i) on your local machine, (ii) in a Docker container, and (iii) on Google Colab. Note that we specifically selected these demos as to replicate some real recommendation tasks involving multimodal features.

5.1 Demo 1: visual + textual items features

Fashion recommendation is probably one of the most popular task involving multimodal features to describe items. Generally, fashion products come with images (i.e., visual) and descriptions (i.e., textual) which may captivate the attention of the customer.

Input data. We use a small fashion dataset where each item has its own image and other metadata such as gender, category, colour, season, and product title. As for the visual modality, we save a subsample of 100 random images from the dataset in jpeg format; as for the textual modality, we produce for each of these items a description obtained as the combination of all the metadata fields from above, and store it into a tsv file where the first and second columns map item ids and descriptions, respectively. Note that, if no item column name is provided, DUCHO selects, by default, the last column as the one holding the items' descriptions.

Extraction. In terms of extraction models, we adopt VGG19 and Xception for the product images, and Sentence-BERT pre-trained for semantic textual similarity for the descriptions. For each extraction model, we select the extraction layer, the pre-processing procedures, and the library where the deep network should be retrieved from.

Output. Through the configuration file, we set DUCHO to save the visual and textual embeddings to custom folders, where each embedding is a numpy array whose filename corresponds to the item name from the original input data. Additionally, DUCHO keeps track of the log file in a dedicated folder within the project.

³<https://hub.docker.com/r/sisinfalpoliba/ducho>.

⁴<https://github.com/sisinfal/Ducho/tree/main/demos>.

5.2 Demo 2: audio + textual items features

When it comes to recommending songs to users, audio and textual features may enhance the representation of each song, where the former are structured as a waveform, the latter as sentences referring, for instance, to the music genre of the song.

Input data. We use a small music genres dataset where each item comes with the binary representation of its waveform (we save it as wav audio track) and its music genre (we interpret it as textual song description and save it into a tsv file similarly to the previous demo). Given the heavy computational costs deep learning-based audio extractors require, we decide to select a small subset of the input songs (i.e., 10) just for the purpose of this demo.

Extraction. For the extraction of audio features we exploit Hybrid Demucs pre-trained for the task of music source separation. As for the textual extraction, we re-use the same deep neural model from the previous demo, since we are not interested in extracting other specific high-level features from music genres.

Output. Once again, we use the configuration file to specify the output folders for both the audio and textual embeddings. Please note that the extraction of audio features might take some time depending on the machine you are running DUCHO on, as the deep audio extractor might require high computational resources to run.

5.3 Demo 3: textual items/interactions features

Online platforms usually allow customers to express reviews and comments about the products they have enjoyed to share their experience with other potentially-interested customers. In an e-commerce scenario, items may come with textual descriptions of the product characteristics (as seen in Demo 1). However, textual reviews of users commenting on those items may also be involved. Unlike most existing literature works, which usually refer to both sources of information as items' representations, we decide to conceptually distinguish between items- and interactions (i.e., user-item)-side representations for the former and the latter, respectively.

Input data. We adopt the widely-popular Amazon recommendation dataset where each user's purchase keeps track of metadata such as customer/product ids, the review text, the rating, and the purchase date. In a similar manner to the other demos, we retain only a small subset of the original dataset including 100 reviews and the corresponding product descriptions (obtained as the concatenation of their product title and category). Specifically, we save descriptions and reviews into separate tsv files where the former follow the same format as Demo 1 and Demo 2, while the latter maps user/item ids to review texts. Note that the number of products does not correspond to the number of user-item interactions as we only consider the set of unique interacted items. While DUCHO extracts, by default, description/interaction texts from the last column of the tsv file, here we provide explicit column names to tell DUCHO where to retrieve product descriptions and user reviews from the respective tsv files.

Extraction. While for the items' descriptions we use again the same sentences encoder as in Demo 1 and 2, we decide to extract textual features from users' reviews through a multilingual BERT-based model pre-trained on customers' reviews and specify the task of sentiment analysis for this model.

Output. Textual item features are saved to numpy arrays whose filenames are the item ids. Conversely, the textual interaction features are saved under the filename obtained from user and item ids to provide a unique pointer to each review.

6 CONCLUSION AND FUTURE WORK

In this paper we propose **DUCHO**, a framework for extracting high-level features for multimodal-aware recommendation. Our main purpose is to provide a unified and shared tool to support practitioners and researchers in processing and extracting multimodal features used as side information in recommender systems. Concretely, DUCHO involves three main modules: Dataset, Extractor, and Runner. The multimodal extraction pipeline can be highly customized through a Configuration component that allows the setup of the modalities involved (i.e., audio, visual, textual), the sources of multimodal information (i.e., items and/or user-item interactions), and the pre-trained models along with their main extraction parameters. To show how DUCHO works in different scenarios and settings, we propose three demos accounting for the extraction of (i) visual/textual items features, (ii) audio/textual items features, and (iii) textual items/interactions features. They can be run locally, on Docker (as we also dockerize DUCHO), and on Google Colab. As future directions, we plan to: (i) adopt all available backends (i.e., TensorFlow, PyTorch, and Transformers) to extract features for all modalities; (ii) implement a general extraction model interface allowing the users to follow the same naming/indexing scheme for all pre-trained models and their extraction layers; (iii) integrate the extraction of low-level multimodal features.

ACKNOWLEDGMENTS

This work was partially supported by the following projects: Secure Safe Apulia, MISE CUP: I14E20000020001 CTEMT - Casa delle Tecnologie Emergenti Comune di Matera, CT_FINCONS_III, OVS Fashion Retail Reloaded, LUTECH DIGITALE 4.0, KOINÈ.

REFERENCES

- [1] Vito Walter Anelli, Yashar Deldjoo, Tommaso Di Noia, Eugenio Di Sciascio, Antonio Ferrara, Daniele Malitesta, and Claudio Pomo. 2022. Reshaping Graph Recommendation with Edge Graph Collaborative Filtering and Customer Reviews. In *DL4SR@CIKM (CEUR Workshop Proceedings, Vol. 3317)*. CEUR-WS.org.
- [2] Yashar Deldjoo, Tommaso Di Noia, Daniele Malitesta, and Felice Antonio Merra. 2021. A Study on the Relative Importance of Convolutional Neural Networks in Visually-Aware Recommender Systems. In *CVPR Workshops*. Computer Vision Foundation / IEEE, 3961–3967.
- [3] Yashar Deldjoo, Tommaso Di Noia, Daniele Malitesta, and Felice Antonio Merra. 2022. Leveraging Content-Style Item Representation for Visual Recommendation. In *ECIR (2) (Lecture Notes in Computer Science, Vol. 13186)*. Springer, 84–92.
- [4] Daniele Malitesta, Giandomenico Cornacchia, Claudio Pomo, and Tommaso Di Noia. 2023. Disentangling the Performance Puzzle of Multimodal-aware Recommender Systems. In *EvalRS@KDD (CEUR Workshop Proceedings, Vol. 3450)*. CEUR-WS.org.
- [5] Weiqing Min, Shuqiang Jiang, and Ramesh C. Jain. 2020. Food Recommendation: Framework, Existing Solutions, and Challenges. *IEEE Trans. Multim.* 22, 10 (2020), 2659–2671.
- [6] Sergio Oramas, Oriol Nieto, Mohamed Sordo, and Xavier Serra. 2017. A Deep Multimodal Approach for Cold-start Music Recommendation. In *DLRS@RecSys*. ACM, 32–37.
- [7] Aghiles Salah, Quoc-Tuan Truong, and Hady W. Lauw. 2020. Cornac: A Comparative Framework for Multimodal Recommender Systems. *J. Mach. Learn. Res.* 21 (2020), 95:1–95:5.
- [8] Zixuan Yi, Xi Wang, Iadh Ounis, and Craig MacDonald. 2022. Multi-modal Graph Contrastive Learning for Micro-video Recommendation. In *SIGIR*. ACM, 1807–1811.